DENTAL CLINIC DEMO

Backend architecture, API integrations & implementation roadmap

This document focuses only on the backend required for a small dental clinic website demo: what the server must do, which external APIs it connects to, how the components communicate, and the order in which to build and test them.

> **Recommended stack: Node.js + Express REST API · MongoDB · server-side email provider integration (Resend, Brevo, or SendGrid) · secure admin authentication. WhatsApp and Google Maps can start as links, not backend API integrations.**

1. What the backend is responsible for

- **Receive requests:** accept appointment submissions from the patient website through a REST API.
- **Validate and protect:** validate fields on the server, limit request sizes, rate-limit public endpoints, and reduce spam.
- **Store data:** save appointment requests and clinic/service content in MongoDB.
- **Notify staff:** call an email provider from the server when a new appointment request is saved.
- **Support the admin dashboard:** authenticate staff, return appointment records only to authorized users, and allow status changes.
- **Handle failures:** return consistent HTTP status codes, log operational errors safely, and avoid reporting a notification as successful when it failed.

2. Backend architecture

COMPONENT	RESPONSIBILITY	CONNECTS TO
React patient website	Sends appointment requests and loads public clinic/services data.	Express REST API over HTTPS
Express API server	Routes, validation, authentication, authorization, rate limits, business logic, and error handling.	MongoDB and email provider
Service/business logic layer	Creates appointment records, updates statuses, and triggers notifications.	Database repository and email service module
MongoDB	Stores clinic data, services, users/admin accounts, and appointment requests.	Accessed only by backend; never directly by browser
Email provider API	Delivers a new-request notification to clinic staff and optionally a patient acknowledgement.	Called server-side with secret API key
Admin dashboard	Lists requests and lets staff update status after login.	Protected Express admin endpoints

Request flow

Patient form → **POST /api/appointments** → server validation and spam controls → **MongoDB save** → email provider notification → success response to the website. Staff then sign in to the admin dashboard, load requests through protected endpoints, and update statuses.

Important: the first version is an appointment-request workflow. A request should default to **Pending**; the website should not imply that a time is confirmed until clinic staff confirms it.

3. API integrations and what the backend needs

Integration	Backend work required	MVP decision
Appointment API	Create REST endpoint, validate request, save record, return request ID and safe response.	**Build now.** Core of the demo.
MongoDB	Create models/collections, indexes where useful, environment-based connection, and backup plan.	**Build now.** Stores requests and app data.
Email provider (Resend / Brevo / SendGrid)	Create server-side mail module; use verified sender; store API key in environment; handle provider errors.	**Build now.** Staff notification is recommended.
Admin authentication	Login/logout, password hashing, secure session or token handling, and authorization middleware on every admin route.	**Build now** if dashboard manages real records.
WhatsApp click-to-chat	No server integration needed for basic contact. Website opens a URL containing the clinic number and optional prefilled message.	Add as a frontend link. Business Platform API only for automation.
Google Maps	No backend integration needed for a directions link or basic embed. Keep clinic address in public configuration/data.	Add a link/embed. Use Maps APIs only for advanced features.
SMS / WhatsApp Business API	Would require provider credentials, templates/consent where applicable, delivery/error handling, and opt-out rules.	Defer until automated reminders are required.
Calendar / live availability	Would require calendar integration, timezone handling, conflict checks, and atomic reservation logic.	Defer; manual confirmation is simpler for demo.
Analytics	Usually frontend configuration plus privacy-conscious event definitions; do not send names, phone numbers, or form text.	Optional; not required for backend MVP.

4. Proposed REST API endpoints

METHOD & ROUTE	ACCESS	PURPOSE / EXPECTED BEHAVIOUR
GET /api/clinic	Public	Return approved public clinic details: name, address, phone, hours, map and WhatsApp links.
GET /api/services	Public	Return available treatments and descriptions.
POST /api/appointments	Public + rate limit	Validate request; save as Pending; trigger staff email; return request ID and generic success message.
POST /api/admin/login	Public login route	Verify credentials and establish a secure session or issue a short-lived token.
POST /api/admin/logout	Admin only	Invalidate the current session or token where supported.
GET /api/admin/appointments	Admin only	Return paginated appointment list; optionally filter by status and date.
GET /api/admin/appointments/:id	Admin only	Return one appointment record to authorized staff.
PATCH /api/admin/appointments/:id	Admin only	Update status or staff-only note after validating allowed fields and values.

Suggested appointment record

id (server-generated), **createdAt**, **status** (Pending by default), **patientName**, **phone**, **service**, **preferredDate**, **preferredTime**, **message** (optional, short), and **staffNote** (admin-only). Collect only fields the clinic needs. Tell patients not to enter detailed medical information in the general enquiry box.

5. Suggested backend project structure

PATH	RESPONSIBILITY
`src/`	Application source
`src/server.js`	Starts server and handles graceful shutdown
`src/app.js`	Express app, middleware, routes, and central error handler
`src/config/`	Environment configuration and database connection
`src/routes/`	Public routes and admin routes
`src/controllers/`	Translate HTTP requests into service calls and responses
`src/services/appointments.service.js`	Appointment business rules and status updates
`src/services/email.service.js`	Email provider integration and notification templates
`src/models/`	MongoDB schemas/models: Appointment, AdminUser, Clinic, Service
`src/middleware/auth.js`	Authentication and admin authorization
`src/middleware/validation.js`	Request validation, rate limits, and spam controls
`src/utils/`	Shared errors, response helpers, and safe logging
`.env`	Secrets and environment-specific settings; never commit this file
`.env.example`	Names of required environment variables without real secrets
`tests/`	API, authorization, validation, and notification tests

Key implementation decisions

- Keep route handlers thin; put appointment rules and email delivery in service modules.
- Use one consistent response format and central error handler. Never return stack traces to public clients.
- Keep email provider keys, database URI, session secrets, and admin bootstrap secrets on the server only.
- Do not connect the browser directly to MongoDB or expose internal database credentials.
- For a small MVP, a modular monolith (one Express application with clear modules) is simpler than microservices.

6. Backend implementation roadmap

PHASE	IMPLEMENTATION TASKS	DONE WHEN
1. Project setup	Create Node project; configure Express, environment variables, linting, central errors, and health endpoint.	Server starts reliably; secrets are not committed.
2. Database	Connect MongoDB; create Appointment, AdminUser, Clinic, and Service models; add indexes if useful.	App connects safely and persists test records.
3. Public read APIs	Implement GET /api/clinic and GET /api/services with only public fields.	Website can load clinic and service content.
4. Appointment flow	Implement POST /api/appointments, server-side validation, spam controls, Pending status, and safe responses.	Valid request saves once; invalid input is rejected.
5. Email integration	Configure provider account and verified sender; create email service; handle provider errors and logs.	Test request triggers staff notification; failure is observable and handled.
6. Admin security	Create secure admin bootstrap, password hashing, login/logout, auth middleware, and route authorization.	Private endpoints reject unauthenticated/unauthorized requests.
7. Admin APIs	Build list/detail/filter endpoints and validated status updates; paginate results.	Staff can manage requests without accessing unrelated data.
8. Integration & deployment	Connect frontend; configure HTTPS, production env vars, database access, backups, monitoring, and deployment.	End-to-end test passes in a demo environment.

7. Security and operational requirements

- **Transport and secrets:** use HTTPS in deployment; store credentials in environment variables or a secrets manager.
- **Input safety:** validate type, length, allowed values, and date formats on the server; sanitize output where relevant.
- **Access control:** protect every admin endpoint, not just the dashboard page. Use secure, HttpOnly cookies with appropriate SameSite settings if using sessions; consider CSRF protection.
- **Password handling:** hash passwords with a modern password-hashing library; never store plain-text passwords or commit default credentials.
- **Abuse controls:** rate-limit appointment submissions and login attempts; use a honeypot or CAPTCHA if spam becomes an issue.
- **Data minimization:** collect only what is needed for follow-up; set a retention/deletion process and restrict database access.
- **Logging:** log request IDs and technical errors, but avoid logging full phone numbers, messages, passwords, tokens, or unnecessary patient details.
- **Email reliability:** do not expose provider keys in React; do not make the appointment save depend on a successful email delivery. Save the request first, then attempt notification and handle/retry failures.
- **Deployment:** configure database backups, error monitoring, environment separation, and a tested restore procedure before using real clinic data.

8. End-to-end test checklist

- ■ Valid patient submission creates one MongoDB record with a server-generated ID, timestamp, and Pending status.
- ■ Missing required fields, invalid dates, oversized text, and malformed contact details are rejected with clear errors.
- ■ Duplicate clicks or retries do not unintentionally create duplicate records; add an idempotency strategy if needed.
- ■ Email notification is sent on a successful test; if the email provider fails, the request remains saved and the failure is logged safely.
- ■ Unauthenticated requests to admin endpoints receive an authorization error; authorized staff can list and update records.
- ■ A user cannot update arbitrary fields, change permissions, or access staff-only notes through public endpoints.
- ■ API secrets are absent from frontend bundles, repository history, and public responses.
- ■ WhatsApp and Maps links point to the correct clinic contact details; no backend API key is needed for the basic link use case.
- ■ Production settings use HTTPS, restricted database access, backups, and separate development/production credentials.

9. Explicitly out of scope for the first backend

Do not add live calendar booking, SMS/WhatsApp automation, payment processing, patient accounts, AI chat, or practice-management software sync in the first iteration unless the clinic specifically needs them. These add provider configuration, consent and privacy considerations, operational failure cases, and extra testing. Start with request submission, storage, email notification, and protected staff management.

This is an implementation blueprint, not legal advice. Before collecting real patient data, confirm applicable privacy obligations, provider terms, and the clinic's data-retention practices.